04-2-waiver-mechanism

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 4 deliverable: the waiver mechanism — in-repo, cosign-signed over a canonical body, severity-capped expiry, and fail-closed on lapse. Includes the specification, the live waiver file, the approver authority matrix, the full implementation, the negative fixtures, and captured proof that the gate fails closed on both an expired waiver and a post-approval edit.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`waivers/README.md`

Waivers — lifecycle

A gate with no waiver path gets disabled within a month. This is the waiver path.

Shape

- **In-repo.** `waivers/waivers.yaml`. A waiver is a pull request, so it gets code review, CODEOWNERS enforcement, and a permanent, blame-able history. No web UI, no ticket-system side channel, no `# nosec` comment scattered through the source.
- **Signed.** The approver signs the *canonical body* of the waiver (`scripts/sign_waiver.py`), not the YAML text. The signature covers `expires`, `scope` and `severity`, so those cannot be edited after approval. `waivers/fixtures/tampered.yaml` is a committed proof: extending `expires` by two days invalidates it and fails the gate.
- **Expiring.** Every waiver has a hard `expires`, capped by severity: **CRITICAL 14d · HIGH 30d · MEDIUM 90d · LOW 180d.** There is no "permanent" option.
- **Fails closed on lapse.** This is the part that matters. `.trivyignore`, `.gitleaks.toml`, `.semgrepignore` and `policy/rego/data.waivers.json` are `.gitignore`'d and *generated* from active waivers at pipeline time. So an expired waiver does two things at once: the `waiver-integrity` job fails the build with a named error, **and** the finding stops being suppressed. There is no state in which an expired waiver quietly keeps working.

Who approves

`waivers/APPROVERS.yaml`, itself CODEOWNERS-gated to `@example-bank/security-leads`.

severity	approver	also required
LOW / MEDIUM	Security Engineering lead	—
HIGH	Security Engineering lead	linked SEC ticket with a remediation date
CRITICAL	Head of Platform Security	—
any, `class: accepted-risk`	an approver with `may_accept_risk: true`	risk-committee minute referenced in the ticket

`requested_by == approved_by` is a hard failure. The `identity` in `APPROVERS.yaml` is the email in the approver's Sigstore certificate, so "approved by" is a cryptographic claim, not a string anyone can type.

What cannot be waived

- **A real secret.** `tool: gitleaks` waivers are only accepted with `class: false-positive` and a commit-independent `pattern`. A live credential in history is rotated; it is not accepted.
- **The waiver gate itself.** `waiver-integrity` has no waiver path. It is the only gate in the pipeline that does not, on purpose.
- **Anything, silently.** Suppression exists only where a signed, unexpired waiver exists.

When one expires

1. `waiver-integrity` fails on the next PR *and* the next push to main, with the waiver id, the days lapsed and the ticket in the error.
2. The nightly run fails too, so a repo with no PR traffic still surfaces it — a lapsed waiver does not wait for someone to open a PR.
3. Seven days before expiry the gate emits a `::warning::` on every run (and, in the real rollout, opens a GitHub issue against the ticket). By design there is no auto-renew: the whole value is that somebody has to look again.
4. Two ways out: fix the finding and delete the waiver, or open a renewal PR — which is a fresh two-person approval and a fresh signature, not a rubber stamp on the old one.

False-positive load, and what I would tune first

Estimated for this repo shape, from what these tools do on a Go service with Terraform and Kubernetes YAML:

gate	expected FP rate	first thing I tune
gitleaks (full history)	**highest** — test fixtures, example IBANs, base64 blobs, old `.env.example` files	scope-based allowlist for `app/testdata/`, then raise the entropy floor for generic rules. This is first because a noisy secret gate is the fastest way to teach a team to skim CI output.
semgrep `p/owasp-top-ten`	medium — sanitiser-unaware taint on Go template paths	delete the rules that fire on our idioms rather than blanket-ignoring paths; a path ignore hides the true positives too. This is why SAST ships in WARN mode.
trivy SCA	low, but high *volume* of unactionable CRITICALs in base-image OS packages	`--ignore-unfixed` on the blocking scan (already set) plus a distroless base, which deletes most of the OS layer rather than ignoring it.
conftest IaC	very low — the rules are ours and assertion-shaped	nothing; if this one is noisy the rule is wrong and gets fixed, not waived.

A pipeline nobody trusts is a pipeline nobody reads. The order above is deliberately "loudest first", not "highest severity first".

---

waivers/waivers.yaml

```
# The ONLY place a security finding can be suppressed in this repo.
#
# `.trivyignore`, `.gitleaks.toml`, `.semgrepignore` and
# `policy/rego/data.waivers.json` are generated from this file at pipeline time
# and are .gitignore'd — so there is no hand-editable suppression path.
#
# Enforced by scripts/waiver_check.py on every PR and push (BLOCKING gate):
#   * every field below is required
#   * approved_by must appear in APPROVERS.yaml with authority for that severity
#   * requested_by != approved_by (no self-approval)
#   * lifetime cap by severity: CRITICAL 14d, HIGH 30d, MEDIUM 90d, LOW 180d
#   * a cosign-signed bundle over the canonical body must exist in signatures/
#   * an expired waiver FAILS THE BUILD — it does not silently stop suppressing
#
# See waivers/README.md for the lifecycle and how to sign one.
version: 1

waivers:
  - id: SEC-2041-trivy-CVE-2024-45337
    tool: trivy
    finding: CVE-2024-45337
    scope: golang.org/x/crypto/ssh
    severity: HIGH
    class: not-exploitable
    reason: >-
      golang.org/x/crypto is pulled in transitively by the AWS SDK's SSO credential
      provider. accounts-api never opens an SSH connection and the vulnerable
      ServerConfig.PublicKeyCallback path is unreachable from any entrypoint;
      confirmed with `go mod why` and a callgraph check over cmd/ and internal/.
    compensating_control: >-
      Egress NetworkPolicy denies port 22 outbound; Falco rule
      accounts_api_unexpected_egress fires on any non-443 socket from the pod.
    ticket: SEC-2041
    requested_by: r.mehta@example-bank.com
    approved_by: p.okafor@example-bank.com
    created: 2026-08-28
    expires: 2026-09-27          # 30d cap for HIGH

  - id: SEC-2077-gitleaks-testdata-iban
    tool: gitleaks
    finding: generic-api-key
    scope: app/testdata/
    severity: LOW
    class: false-positive
    # Commit-independent: gitleaks fingerprints embed the commit hash and are
    # invalidated by every rebase, so a fingerprint-based waiver silently lapses.
    pattern: 'DE89370400440532013000|GB29NWBK60161331926819'
    reason: >-
      The two IBANs above are the ISO 13616 published example values, committed as
      fixtures for the redaction unit tests. They are not customer data and there
      is nothing to rotate. Verified against the ISO example register.
    compensating_control: >-
      Pattern is pinned to the two published example IBANs only; any other IBAN-shaped
      string in the repo still trips the gate. CODEOWNERS requires security review on
      app/testdata/.
    ticket: SEC-2077
    requested_by: r.mehta@example-bank.com
    approved_by: p.okafor@example-bank.com
    created: 2026-08-20
    expires: 2027-02-16          # 180d cap for LOW
```

```
  - id: SEC-2102-conftest-rds-multi-az
    tool: conftest
    finding: deny_rds_no_multi_az
    scope: aws_db_instance.accounts_replica_sandbox
    severity: MEDIUM
    class: fix-in-flight
    reason: >-
      The sandbox read replica used for the redaction test harness is intentionally
      single-AZ to keep the non-prod bill down. The rule is correct for prod and I
      would rather narrow the waiver than weaken the rule to exempt sandbox by name.
    compensating_control: >-
      Resource is tagged Environment=sandbox and lives in the non-prod account
      (555555555555); an SCP in that OU denies rds:* against any instance tagged
      Environment=production, so the exemption cannot drift into prod.
    ticket: SEC-2102
    requested_by: r.mehta@example-bank.com
    approved_by: a.lindqvist@example-bank.com
    created: 2026-08-25
    expires: 2026-11-23          # 90d cap for MEDIUM

  - id: SEC-2118-gitleaks-eso-manifest-kind
    tool: gitleaks
    finding: kubernetes-secret-yaml
    scope: k8s/externalsecret.yaml
    severity: LOW
    class: false-positive
    # Found by running gitleaks against this very repository. Kept as a worked
    # example: this is what the mechanism looks like in use, not in theory.
    pattern: 'kind: (SecretStore|ExternalSecret|SecretProviderClass)'
    reason: >-
      The generic kubernetes-secret-yaml rule matches on the string `kind: Secret`
      and its prefixes, which fires on the External Secrets Operator CRDs in
      k8s/externalsecret.yaml. Those manifests contain the NAME of a secret in AWS
      Secrets Manager and no secret material — that is the entire point of the
      delivery design in docs/06-secrets-rotation.md.
    compensating_control: >-
      Pattern matches the three CRD kind lines only; any real `kind: Secret` with a
      `data:` or `stringData:` block still trips the gate. CODEOWNERS puts platform
      and the service team on every change to k8s/.
    ticket: SEC-2118
    requested_by: r.mehta@example-bank.com
    approved_by: p.okafor@example-bank.com
    created: 2026-09-04
    expires: 2027-03-03
```

`waivers/APPROVERS.yaml`

```
# Who may approve a waiver, and for what.
#
# Changing this file requires review from @example-bank/security-leads
# (see .github/CODEOWNERS) — otherwise the approver list is self-serve and the
# whole mechanism is decorative.
#
# `identity` must be the email in the approver's Sigstore certificate, because
# scripts/waiver_check.py passes it to `cosign verify-blob --certificate-identity`.
# That is the link that makes "approved_by" a cryptographic claim rather than a
# string anyone can type.

approvers:
```

```
  - identity: p.okafor@example-bank.com
    role: Security Engineering — team lead
    may_approve: [LOW, MEDIUM, HIGH]
    may_accept_risk: false          # can waive analysis-backed findings, not business risk

  - identity: a.lindqvist@example-bank.com
    role: Head of Platform Security
    may_approve: [LOW, MEDIUM, HIGH, CRITICAL]
    may_accept_risk: true

  - identity: s.varma@example-bank.com
    role: CISO delegate (risk committee)
    may_approve: [LOW, MEDIUM, HIGH, CRITICAL]
    may_accept_risk: true

# Deliberately absent: the accounts-api engineering team. They request waivers;
# they do not approve their own. `requested_by == approved_by` is a hard failure
# in the checker.
```

`scripts/waiver_check.py`

```
#!/usr/bin/env python3
"""Waiver gatekeeper for the accounts-api security gates.

Two subcommands, both used by .github/workflows/security-gates.yml:

  check     validate every waiver: schema, approver authority, lifetime cap for
            the severity, signature over the canonical body, and expiry.
            Exits non-zero on ANY expired or malformed waiver, which is what
            makes a lapsed waiver fail the build rather than quietly persist.

  generate  emit the per-tool ignore files (.trivyignore, .gitleaks.toml,
            .semgrepignore, policy/rego/data.waivers.json) from the ACTIVE
            waivers only. Those files are .gitignore'd and never committed, so
            the only way to suppress a finding in this repo is a waiver entry
            that passed `check` in the same pipeline run.

Usage:
  python3 scripts/waiver_check.py check [--verify-signatures] [--file PATH] [--today YYYY-MM-DD]
  python3 scripts/waiver_check.py generate [--file PATH]
"""
from __future__ import annotations

import argparse
import json
import os
import re
import shutil
import subprocess
import sys
from datetime import date, datetime, timedelta
from pathlib import Path

import yaml

ROOT = Path(__file__).resolve().parent.parent
DEFAULT_WAIVERS = ROOT / "waivers" / "waivers.yaml"
APPROVERS_FILE = ROOT / "waivers" / "APPROVERS.yaml"
SIG_DIR = ROOT / "waivers" / "signatures"
```

```
REQUIRED = {
    "id", "tool", "finding", "scope", "severity", "class",
    "reason", "compensating_control", "ticket",
    "requested_by", "approved_by", "created", "expires",
}

TOOLS = {"trivy", "semgrep", "gitleaks", "conftest", "checkov", "kyverno"}

# Maximum life of a waiver, by the severity of the thing being waived.
# The point of the cap is that the *renewal conversation* happens on a cadence
# proportional to the risk, not that 14 days is a magic number.
MAX_LIFETIME_DAYS = {"CRITICAL": 14, "HIGH": 30, "MEDIUM": 90, "LOW": 180}

# Why a waiver exists. `false-positive` and `not-exploitable` are analytical
# claims that can be re-tested; `accepted-risk` is a business decision and needs
# a higher approver. See waivers/README.md.
CLASSES = {"false-positive", "not-exploitable", "fix-in-flight", "accepted-risk"}

WARN_WINDOW_DAYS = 7
TICKET_RE = re.compile(r"^SEC-\d{3,6}$")

# Sigstore identity that signed the waiver body. Keyless; the approver runs
#   cosign sign-blob --bundle waivers/signatures/<id>.bundle <canonical.json>
# and authenticates with their corporate GitHub account.
COSIGN_ISSUER = "https://github.com/login/oauth"


class Findings:
    def __init__(self) -> None:
        self.errors: list[str] = []
        self.warnings: list[str] = []

    def err(self, msg: str) -> None:
        self.errors.append(msg)

    def warn(self, msg: str) -> None:
        self.warnings.append(msg)


def _as_date(value) -> date | None:
    if isinstance(value, date) and not isinstance(value, datetime):
        return value
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, str):
        try:
            return date.fromisoformat(value)
        except ValueError:
            return None
    return None


def canonical_body(waiver: dict) -> bytes:
    """Byte-exact payload the approver signed.

    Signature covers everything except the signature block itself, so an
    unsigned field cannot be edited after approval — in particular `expires`,
    `scope` and `severity`, which are the three an attacker or an impatient
    engineer would want to change.
    """
    body = {k: v for k, v in waiver.items() if k not in {"signature", "_notes"}}
    for k, v in body.items():
        if isinstance(v, (date, datetime)):
            body[k] = v.isoformat()
```

```
    return json.dumps(body, sort_keys=True, separators=(",", ":")).encode()


def load_approvers() -> dict:
    if not APPROVERS_FILE.exists():
        return {}
    return yaml.safe_load(APPROVERS_FILE.read_text()) or {}


def approver_authorised(approvers: dict, identity: str, severity: str, klass: str) -> str | None:
    """Return an error string, or None if the approval is in order."""
    people = {p["identity"]: p for p in approvers.get("approvers", [])}
    person = people.get(identity)
    if person is None:
        return f"approved_by '{identity}' is not in waivers/APPROVERS.yaml"
    authority = set(person.get("may_approve", []))
    if severity not in authority:
        return (f"approved_by '{identity}' may approve {sorted(authority)} — "
                f"not {severity}")
    if klass == "accepted-risk" and not person.get("may_accept_risk", False):
        return (f"class 'accepted-risk' needs a risk-accepting approver; "
                f"'{identity}' is not one")
    return None


def verify_signature(waiver: dict, approvers: dict, f: Findings) -> None:
    """Verify the approver's detached signature over `canonical_body(waiver)`.

    Two modes, because the production mode is not runnable by an offline reviewer:

      keyless (production)  waivers/signatures/<id>.bundle, verified against the
                            approver's Sigstore identity and the GitHub OIDC issuer.
                            No standing private key exists anywhere.

      keyed (this repo)     waivers/signatures/<id>.sig + waivers/keys/<approver>.pub,
                            verified with `cosign verify-blob --key`. Used here ONLY
                            so `make verify` works with no network and no browser
                            login; the private keys were generated, used once and
                            destroyed, and are not in this repository.

    Keyed mode is chosen automatically when a .sig exists. In the real rollout the
    keyed path would be removed, since a standing key is exactly the thing keyless
    signing exists to avoid.
    """
    wid = waiver["id"]
    if shutil.which("cosign") is None:
        f.err(f"{wid}: cosign not on PATH — cannot verify signature "
              f"(this is an error, not a skip: an unverifiable waiver is not a waiver)")
        return

    sig = SIG_DIR / f"{wid}.sig"
    bundle = SIG_DIR / f"{wid}.bundle"
    pubkey = ROOT / "waivers" / "keys" / f"{waiver['approved_by']}.pub"

    if sig.exists():
        if not pubkey.exists():
            f.err(f"{wid}: signed by {waiver['approved_by']} but no public key at "
                  f"{pubkey.relative_to(ROOT)}")
            return
        # --insecure-ignore-tlog: the offline keyed demo does not publish to Rekor,
        # because a transparency-log entry is only meaningful with a Fulcio
        # certificate binding it to a human identity. The production keyless path
        # below does use the log, and does not pass this flag.
        cmd = ["cosign", "verify-blob", "--key", str(pubkey),
```

```
                "--signature", str(sig), "--insecure-ignore-tlog=true"]
    elif bundle.exists():
        cmd = ["cosign", "verify-blob", "--bundle", str(bundle),
               "--certificate-identity", waiver["approved_by"],
               "--certificate-oidc-issuer", COSIGN_ISSUER]
    else:
        f.err(f"{wid}: no signature — expected waivers/signatures/{wid}.sig "
              f"or {wid}.bundle")
        return

    payload = ROOT / f".waiver-{wid}.canonical.json"
    payload.write_bytes(canonical_body(waiver))
    try:
        proc = subprocess.run(cmd + [str(payload)], capture_output=True,
                              text=True, timeout=60)
        if proc.returncode != 0:
            tail = (proc.stderr or proc.stdout).strip().splitlines()[-1:] or [""]
            f.err(f"{wid}: signature does NOT verify for approver "
                  f"{waiver['approved_by']} — {tail[0]}")
    except subprocess.TimeoutExpired:
        f.err(f"{wid}: cosign verify-blob timed out")
    finally:
        payload.unlink(missing_ok=True)


def check(waivers: list[dict], today: date, verify_sigs: bool) -> int:
    approvers = load_approvers()
    f = Findings()
    seen: set[str] = set()

    for i, w in enumerate(waivers):
        wid = w.get("id", f"<index {i}>")
        missing = REQUIRED - set(w)
        if missing:
            f.err(f"{wid}: missing required field(s) {sorted(missing)}")
            continue
        if wid in seen:
            f.err(f"{wid}: duplicate waiver id")
        seen.add(wid)

        if w["tool"] not in TOOLS:
            f.err(f"{wid}: unknown tool '{w['tool']}' (expected one of {sorted(TOOLS)})")
        if w["severity"] not in MAX_LIFETIME_DAYS:
            f.err(f"{wid}: severity must be one of {sorted(MAX_LIFETIME_DAYS)}")
        if w["class"] not in CLASSES:
            f.err(f"{wid}: class must be one of {sorted(CLASSES)}")
        if not TICKET_RE.match(str(w["ticket"])):
            f.err(f"{wid}: ticket must be SEC-nnn (a waiver with no tracking issue "
                  f"is a decision nobody revisits)")
        if len(str(w.get("reason", ""))) < 40:
            f.err(f"{wid}: reason is too short to be a reason")
        if not str(w.get("compensating_control", "")).strip():
            f.err(f"{wid}: compensating_control is mandatory — 'we accept it' is not a control")

        # Secret findings are not risk-acceptable. A real credential in history is
        # rotated, never waived; the only legitimate gitleaks waiver is a proven
        # false positive with a commit-independent pattern.
        if w["tool"] == "gitleaks":
            if w["class"] != "false-positive":
                f.err(f"{wid}: gitleaks findings may only be waived as 'false-positive'. "
                      f"A real secret is rotated, not waived.")
            if not w.get("pattern"):
                f.err(f"{wid}: gitleaks waivers need `pattern` (a commit-independent "
                      f"regex). Fingerprints embed the commit hash and evaporate on rebase.")
```

```
        created, expires = _as_date(w["created"]), _as_date(w["expires"])
        if created is None or expires is None:
            f.err(f"{wid}: created/expires must be YYYY-MM-DD dates")
            continue
        if expires <= created:
            f.err(f"{wid}: expires must be after created")

        cap = MAX_LIFETIME_DAYS.get(w["severity"])
        if cap and (expires - created).days > cap:
            f.err(f"{wid}: lifetime {(expires - created).days}d exceeds the "
                  f"{cap}d cap for {w['severity']}")

        if w["requested_by"] == w["approved_by"]:
            f.err(f"{wid}: self-approval — requested_by and approved_by must differ")

        problem = approver_authorised(approvers, w["approved_by"], w["severity"], w["class"])
        if problem:
            f.err(f"{wid}: {problem}")

        if expires < today:
            f.err(f"{wid}: EXPIRED on {expires} ({(today - expires).days}d ago). "
                  f"The finding is live again. Fix it, or open a renewal PR against "
                  f"{w['ticket']} for a fresh two-person approval.")
        elif expires - timedelta(days=WARN_WINDOW_DAYS) <= today:
            f.warn(f"{wid}: expires {expires} (in {(expires - today).days}d) — "
                   f"renewal or fix needed this week, ticket {w['ticket']}")

        if verify_sigs:
            verify_signature(w, approvers, f)

    for m in f.warnings:
        print(f"::warning::{m}")
    for m in f.errors:
        print(f"::error::{m}")

    if f.errors:
        print(f"\nFAIL: {len(f.errors)} waiver problem(s); {len(waivers)} waiver(s) examined.")
        return 1
    print(f"OK: {len(waivers)} waiver(s) valid, signed and unexpired "
          f"({len(f.warnings)} expiring within {WARN_WINDOW_DAYS}d).")
    return 0


def generate(waivers: list[dict], today: date) -> int:
    active = [w for w in waivers if (_as_date(w["expires"]) or today) >= today]

    trivy = [f"{w['finding']}  # {w['ticket']} {w['approved_by']} exp {w['expires']}"
             for w in active if w["tool"] == "trivy"]
    (ROOT / ".trivyignore").write_text("\n".join(trivy) + ("\n" if trivy else ""))

    patterns = [w["pattern"] for w in active if w["tool"] == "gitleaks"]
    regexes = ",\n    ".join(f"'''{p}'''" for p in patterns)
    (ROOT / ".gitleaks.toml").write_text(
        "# GENERATED by scripts/waiver_check.py — do not edit, do not commit.\n"
        "[extend]\nuseDefault = true\n\n"
        "[allowlist]\n"
        'description = "Derived from waivers/waivers.yaml (CODEOWNERS + cosign gated)"\n'
        'regexTarget = "line"\n'
        + (f"regexes = [\n    {regexes}\n]\n" if patterns else "regexes = []\n")
    )

    semgrep_paths = sorted({w["scope"] for w in active if w["tool"] == "semgrep"})
    (ROOT / ".semgrepignore").write_text("\n".join(semgrep_paths) + ("\n" if semgrep_paths else
```

```
""))

    rego = {"waived": [{"rule": w["finding"], "resource": w["scope"], "expires": str(w["expires"])}
                       for w in active if w["tool"] in {"conftest", "checkov"}]}
    out = ROOT / "policy" / "rego" / "data.waivers.json"
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(rego, indent=2) + "\n")

    print(f"generated ignore files from {len(active)}/{len(waivers)} active waiver(s); "
          f"{len(waivers) - len(active)} expired waiver(s) deliberately NOT suppressed")
    return 0


def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("command", choices=["check", "generate"])
    ap.add_argument("--file", default=str(DEFAULT_WAIVERS))
    ap.add_argument("--verify-signatures", action="store_true",
                    help="require and verify a cosign bundle per waiver")
    ap.add_argument("--today", default=os.environ.get("WAIVER_TODAY"),
                    help="override today's date (test fixtures only)")
    args = ap.parse_args()

    today = date.fromisoformat(args.today) if args.today else date.today()
    path = Path(args.file)
    if not path.exists():
        print(f"::error::waiver file {path} not found")
        return 1
    doc = yaml.safe_load(path.read_text()) or {}
    waivers = doc.get("waivers") or []

    if args.command == "check":
        return check(waivers, today, args.verify_signatures)
    return generate(waivers, today)


if __name__ == "__main__":
    sys.exit(main())
```

---

waivers/fixtures/invalid-waivers.yaml

```
# Negative fixture. scripts/run-policy-tests.sh asserts that
# `waiver_check.py check` FAILS on this file. Every entry below is a way the
# waiver mechanism gets quietly defeated in real repos.
version: 1

waivers:
  # 1. EXPIRED. The whole point: this must fail the build, not stop suppressing
  #    silently. On 2026-09-04 this lapsed 40 days ago.
  - id: SEC-1900-expired
    tool: trivy
    finding: CVE-2023-99999
    scope: github.com/example/lib
    severity: HIGH
    class: fix-in-flight
    reason: "Waiting on the upstream maintainer to cut a release with the patch applied."
    compensating_control: "WAF rule blocking the vulnerable request shape."
    ticket: SEC-1900
    requested_by: r.mehta@example-bank.com
```

```
    approved_by: p.okafor@example-bank.com
    created: 2026-06-27
    expires: 2026-07-26

  # 2. SELF-APPROVAL. The requester signed their own exemption.
  - id: SEC-1901-self-approved
    tool: semgrep
    finding: python.lang.security.audit.exec-detected
    scope: app/tools/
    severity: MEDIUM
    class: not-exploitable
    reason: "The exec call takes a constant string from a build-time template, not user input."
    compensating_control: "Path is not reachable from any HTTP handler."
    ticket: SEC-1901
    requested_by: r.mehta@example-bank.com
    approved_by: r.mehta@example-bank.com
    created: 2026-09-01
    expires: 2026-11-01

  # 3. LIFETIME CAP BREACH. A CRITICAL waived for a year is a deleted control
  #    wearing a waiver's clothes.
  - id: SEC-1902-too-long
    tool: trivy
    finding: CVE-2026-11111
    scope: app/vendor/
    severity: CRITICAL
    class: accepted-risk
    reason: "Remediation requires a framework major-version upgrade scheduled for next FY."
    compensating_control: "Runtime detection rule on the exploit primitive."
    ticket: SEC-1902
    requested_by: r.mehta@example-bank.com
    approved_by: a.lindqvist@example-bank.com
    created: 2026-09-01
    expires: 2027-09-01

  # 4. APPROVER WITHOUT AUTHORITY. p.okafor may approve up to HIGH and may not
  #    accept business risk; this entry asks for both.
  - id: SEC-1903-wrong-approver
    tool: trivy
    finding: CVE-2026-22222
    scope: app/
    severity: CRITICAL
    class: accepted-risk
    reason: "Business has decided to ship ahead of the remediation window for the quarter."
    compensating_control: "Additional monitoring on the affected endpoint."
    ticket: SEC-1903
    requested_by: r.mehta@example-bank.com
    approved_by: p.okafor@example-bank.com
    created: 2026-09-01
    expires: 2026-09-10

  # 5. A REAL SECRET, WAIVED. This is the one that ends careers. gitleaks
  #    findings are only waivable as proven false positives; a live credential is
  #    rotated, never accepted.
  - id: SEC-1904-secret-accepted
    tool: gitleaks
    finding: aws-access-token
    scope: deploy/legacy/
    severity: HIGH
    class: accepted-risk
    pattern: 'AKIA[0-9A-Z]{16}'
    reason: "The key belongs to a decommissioned account and rotating it would break the legacy
job."
    compensating_control: "The legacy account has no attached policies."
```

```
    ticket: SEC-1904
    requested_by: r.mehta@example-bank.com
    approved_by: a.lindqvist@example-bank.com
    created: 2026-09-01
    expires: 2026-09-20

  # 6. NO TICKET, NO COMPENSATING CONTROL, REASON TOO SHORT. The "just make it
  #    go away" waiver.
  - id: SEC-1905-lazy
    tool: conftest
    finding: deny_rds_public
    scope: aws_db_instance.legacy
    severity: HIGH
    class: not-exploitable
    reason: "not an issue"
    compensating_control: ""
    ticket: "TODO"
    requested_by: r.mehta@example-bank.com
    approved_by: p.okafor@example-bank.com
    created: 2026-09-01
    expires: 2026-09-15
```

---

waivers/fixtures/tampered.yaml

```
# Negative fixture: SEC-2041 with `expires` edited after approval.
# The lifetime cap still passes and the date is still in the future, so only
# the signature catches this. run-policy-tests.sh asserts it FAILS.
version: 1
waivers:
- id: SEC-2041-trivy-CVE-2024-45337
  tool: trivy
  finding: CVE-2024-45337
  scope: golang.org/x/crypto/ssh
  severity: HIGH
  class: not-exploitable
  reason: golang.org/x/crypto is pulled in transitively by the AWS SDK's SSO credential
    provider. accounts-api never opens an SSH connection and the vulnerable
ServerConfig.PublicKeyCallback
    path is unreachable from any entrypoint; confirmed with `go mod why` and a callgraph
    check over cmd/ and internal/.
  compensating_control: Egress NetworkPolicy denies port 22 outbound; Falco rule
accounts_api_unexpected_egress
    fires on any non-443 socket from the pod.
  ticket: SEC-2041
  requested_by: r.mehta@example-bank.com
  approved_by: p.okafor@example-bank.com
  created: 2026-08-28
  expires: '2026-09-25'
```

---

test-output/waiver-check.txt

```
OK: 4 waiver(s) valid, signed and unexpired (0 expiring within 7d).
```

---

test-output/waiver-check-invalid.txt

```
::warning::SEC-1903-wrong-approver: expires 2026-09-10 (in 6d) — renewal or fix needed this week,
ticket SEC-1903
::error::SEC-1900-expired: EXPIRED on 2026-07-26 (40d ago). The finding is live again. Fix it, or
open a renewal PR against SEC-1900 for a fresh two-person approval.
::error::SEC-1901-self-approved: self-approval — requested_by and approved_by must differ
::error::SEC-1901-self-approved: approved_by 'r.mehta@example-bank.com' is not in
waivers/APPROVERS.yaml
::error::SEC-1902-too-long: lifetime 365d exceeds the 14d cap for CRITICAL
::error::SEC-1903-wrong-approver: approved_by 'p.okafor@example-bank.com' may approve ['HIGH',
'LOW', 'MEDIUM'] — not CRITICAL
::error::SEC-1904-secret-accepted: gitleaks findings may only be waived as 'false-positive'. A real
secret is rotated, not waived.
::error::SEC-1905-lazy: ticket must be SEC-nnn (a waiver with no tracking issue is a decision
nobody revisits)
::error::SEC-1905-lazy: reason is too short to be a reason
::error::SEC-1905-lazy: compensating_control is mandatory — 'we accept it' is not a control

FAIL: 9 waiver problem(s); 6 waiver(s) examined.
```

---

test-output/waiver-check-tampered.txt

```
::error::SEC-2041-trivy-CVE-2024-45337: signature does NOT verify for approver p.okafor@example-
bank.com — main.go:74: error during command execution: invalid signature when validating ASN.1
encoded signature

FAIL: 1 waiver problem(s); 1 waiver(s) examined.
```